UNIVERSITY OF COLOMBO SCHOOL OF COMPUTING
16 MAR 2025
EXAMINATIONS & REGISTRATION

U. C. S. C. LIBRARY UNIVERSITY OF COLOMBO

University of Colombo, Sri Lanka

UCSC *University of Colombo School of Computing*

BACHELOR OF SCIENCE IN INFORMATION SYSTEMS

First Year Examination - Semester II - UCSC AY 21 held in March 2025

IS 1214 - Data Structures and Algorithms

Two (02) Hours

101

Answer All Questions

Instructions to Candidates

1. The medium of instruction and questions is in English.
2. Note that questions appear on both sides of the paper. If a page or a part of this question paper is not printed, please inform the supervisor immediately.
3. This paper has **two (02)** parts across **eleven (11) pages**.
4. MCQ should be marked on the MCQ answer sheet and the essay question on the answer book provided.
5. Write your **index number** clearly on all answer sheets.
6. This paper has two (02) parts, Part I and II. <u>Part I</u> consists of **thirty-five (35) MCQ** questions for **seventy (70) marks**, and <u>Part II</u> consists of an **essay question** for **thirty marks (30)**.
7. Calculators, Mobile Phones, smart watches, or any electronic devices capable of storing and retrieving data are not allowed.
8. All rights reserved.

********** **Multiple Choice Questions Marking Method** **********

- Each MCQ question has five (05) answers with one (01) or more (up to 05) correct answers. Each MCQ question is worth two (02) marks. Suppose a question has two (02) correct answers and three (03) incorrect answers, *+2 marks will be shared between the correct options (1 mark per option), and -2 marks will be shared between incorrect options (-0.67)* per option. If one correct and incorrect answer is marked, the final mark for the question will be **1 + (-0.67) = 0.33**.
- Each question can have a positive or a negative value as the final mark. If a student scores less than zero (0) for an MCQ Question, zero (0) would be considered the mark for that question.

PART I

1. Which of the following are potential problems when using malloc() in C?

   (a) Forgetting to free allocated memory can lead to memory leaks.

   (b) malloc() returns NULL if allocation fails, and not checking it can cause segmentation faults.

   (c) Using sizeof(pointer) instead of sizeof(datatype) may allocate incorrect memory size.

   (d) Allocating memory with malloc() guarantees that the memory is initialized to zero.

   (e) Calling free() on a NULL pointer results in undefined behavior.

2. Which of the following statements regarding dynamic memory allocation in the C programming language are **not** true?

   (a) calloc(...) method dynamically allocates a single large block of memory in the given size.

   (b) calloc(...), malloc(...), and realloc(...) return a void pointer.

   (c) malloc(...) method dynamically allocates a single large block of memory in the given size and initializes the memory block with the default value 'zero.'

   (d) realloc(...) method dynamically increases the memory size allocated by malloc(...) or calloc(...) to the specified amount.

   (e) realloc(...) will initialize the additional memory to be dynamically allocated to garbage values.

3. What happens when you try to pop an element from an empty stack?

   (a) The stack resets itself

   (b) The last element is removed

   (c) The program crashes or an underflow condition occurs

   (d) The stack expands automatically

   (e) The stack converts to a queue

4. Which of the following applications use stacks?

   (a) Function calls in recursion

   (b) Undo operations in a text editor

   (c) Printer management

   (d) Memory allocation for arrays

   (e) Parenthesis matching in expressions

PART I

5. If a stack contains the elements [5, 10, 15, 20] (with 20 at the top), what will be the stack after performing two *pop()* operations?

   (a) $[5, 10]$
   (b) $[10, 15, 20]$
   (c) $[5, 10, 15]$
   (d) $[5, 15]$
   (e) $[5, 10, 20]$

6. Which of the following operations are NOT valid in a stack?

   (a) Push
   (b) Pop
   (c) Enqueue
   (d) Peek
   (e) Dequeue

7. Which of the following is a/are non-linear data structure(s)?

   (a) Arrays
   (b) AVL Trees
   (c) Binary Trees
   (d) Stacks
   (e) Queues

8. Which of the following is/are accurate regarding a stack data structure implementation?

   (a) Linked list-based stacks are of finite size and have to be defined at initialization.
   (b) Stacks can be implemented using both arrays and linked lists.
   (c) When a stack is implemented using a doubly linked list, there is no efficiency concern if the implementation uses the Head or the Tail pointer to perform the push and pop operations.
   (d) When a stack is implemented using a singly linked list, the most efficient implementation is to perform push and pop operations using the Head pointer.
   (e) When a stack is implemented using a singly linked list, the most efficient implementation is to perform push and pop operations using the Tail pointer.

PART I

9. Which data structure is best suited for checking balanced parentheses in an expression?

   (a) Queue
   (b) Stack
   (c) Linked List
   (d) Array
   (e) Binary Tree

10. Which of the following data structures allows O(1) time complexity for both insertion and deletion at both ends?

    (a) Stack
    (b) Queue
    (c) Tree
    (d) Singly Linked List
    (e) Doubly Linked List

11. Which of the following statements about array memory allocation and access are always true?

    (a) Elements of an array are always stored in contiguous memory locations.
    (b) Accessing an element in an array in O(1) time is only possible if the array is indexed.
    (c) When an element is deleted, other elements move and merge create a single array.
    (d) Arrays always grows when the index is accessed for the first time.
    (e) If an array is passed to a function, the function can modify the original array.

12. Which statement(s) regarding the queue data structure are **false**?

    (a) Array implementation of the queue data structure assumes circularity.
    (b) In a linked list implementation, if *addToHead* is considered an *enqueue* operation *deleteFromHead* must be the *dequeue* operation.
    (c) In a linked list implementation, if *addToHead* is considered an *enqueue* operation *deleteFromTail* must be the *dequeue* operation.
    (d) In the array implementation of the queue, the queue is full if and only if the first pointer is at zero and the last pointer is at array length - 1.
    (e) In the array implementation of the queue, the queue is considered full when the last pointer is one plus the first pointer.

PART I

13. In which scenarios does a Doubly Linked List offer a clear advantage over a Singly Linked List?

    (a) Removing any element form a given linked list.
    (b) Efficiently traversing in both forward and backward directions
    (c) Stack implementation with push/pop at the head
    (d) Storing an undo/redo history in an application
    (e) Efficient merging of two sorted linked lists

14. What are the challenges when implementing a Doubly Linked List efficiently?

    (a) Maintaining proper pointer adjustments during insertions and deletions
    (b) Increased memory usage per node
    (c) Ensuring the tail pointer is always correctly updated
    (d) Implementing an efficient circular doubly linked list
    (e) Avoiding memory leaks when deleting nodes

15. Which of the following operations are generally more efficient in a Singly Linked List compared to a Doubly Linked List?

    (a) Inserting a node at the head
    (b) Inserting a node at the tail when a tail pointer is maintained
    (c) Deleting a node given only its reference (except the tail)
    (d) Searching for a node by value
    (e) Reversing the list

16. What is the worst-case scenario(s) for Linear Search?

    (a) The element is found at the first index.
    (b) The element is found in the middle of the list.
    (c) The element is not present in the list.
    (d) The search is performed on a sorted array.
    (e) The element is present at the last index of the list.

PART I

17. What are the characteristics of Linear Search?

    (a) It works on sorted and unsorted data.
    (b) It has a time complexity of O(log n) in the worst case.
    (c) It always scans the entire list before stopping.
    (d) It can be implemented iteratively or recursively.
    (e) The best case occurs when the target element is found at the first index.

18. In which scenarios is Linear Search more efficient than other search algorithms?

    (a) When searching in a small dataset.
    (b) When the dataset is unsorted.
    (c) When searching for an element at a random position in a sorted array.
    (d) When performing a search on a linked list.
    (e) When searching for multiple occurrences of an element in a dataset.

19. In which of the following cases would recursion not be an efficient approach?

    (a) Computing Fibonacci numbers using a the standard recursive method
    (b) Finding the height of a tree
    (c) Solving the Tower of Hanoi problem
    (d) Implementing a simple loop-based operation
    (e) Searching for an element in an unordered array

20. Which of the following statements about recursion are true?

    (a) Every recursive function must have a base case.
    (b) Recursive functions always use more memory than iterative functions.
    (c) A recursive function can call itself multiple times within a single execution.
    (d) Recursion can be replaced by iteration in all cases.
    (e) A recursive function without a base case leads to infinite recursion.

21. What type(s) of recursion is/are calculating Factorial?

    (a) Excessive Recursion
    (b) Indirect Recursion
    (c) Nested Recursion
    (d) Non-Tail Recursion
    (e) Tail Recursion

PART I

22. Which of the following statements is/are true about nested recursion?

    (a) A recursive function where the argument of a recursive call is itself another recursive call.

    (b) A recursive function that always makes exactly two recursive calls.

    (c) Recursive function that can lead to deeply nested calls.

    (d) Recursive function that calls another function, which in turn calls the original function.

    (e) Recursive function where the recursion depth is predictable and linear.

23. What will be the output of the below code given below?

```
int Func(int A, int B) {
    if(A <= 3)
        return 1;
    else
        return A * something(A/B, B);
}
something(18,3);
```

    (a) Code infinitely executes

    (b) 1

    (c) 18

    (d) 108

    (e) 147

Consider an elements 10, 60, 30, 50, 40, 80, 70, 90, 100, 5 and 0 inserted into a binary search tree (BST), AVL tree, and a binary search tree balanced using the DSW algorithm (DSW) in the order specified and answer **Questions 24 to 35**.

24. Which of the following are acceptable print sequences for Depth First Traversal of the Binary Search Tree?

    (a) 00, 05, 10, 30, 40, 50, 60, 70, 80, 90, 100

    (b) 00, 05, 40, 50, 30, 70, 100, 90, 80, 60, 10

    (c) 00, 05, 40, 50, 30, 70, 10, 90, 80, 60, 100

    (d) 10, 00, 05, 30, 60, 40, 50, 100, 70 ,80, 90

    (e) 10, 05, 00, 60, 30, 50, 40, 80, 70, 90, 100

PART I

25. Which of the following are acceptable print sequences for Breadth First Traversal of the Binary Search Tree?

    (a) 10, 60, 05, 30, 80, 00, 50, 70, 90, 40, 100
    (b) 10, 05, 60, 00, 30, 80, 50, 70, 90, 40, 100
    (c) 10, 60, 00, 05, 30, 80, 50, 70, 40, 90, 100
    (d) 10, 05, 60, 00, 30, 40, 50, 70, 90, 80, 100
    (e) 10, 60, 05, 00, 30, 80, 40, 70, 90, 50, 100

26. If delete by merge has been used for the binary tree when deleting the root, what are the maximum and minimum heights of the resulting tree?

    (a) 2
    (b) 3
    (c) 4
    (d) 5
    (e) 6

27. Which of the following is/are the rightmost node of the left subtree of each tree?

    (a) 05
    (b) 40
    (c) 50
    (d) 60
    (e) 70

28. Which of the following is/are accurate regarding the three (03) trees created above?

    (a) The height of the two balanced trees is three or four.
    (b) The tree generated by the DSW algorithm is a complete binary tree.
    (c) The generated AVL tree is a complete binary tree.
    (d) The binary tree generated is a complete binary tree.
    (e) The generated AVL tree is a full binary tree.

PART I

29. What are the roots of the three (03) trees?

    (a) BST - 05, AVL - 30, DSW - 80
    (b) BST - 10, AVL - 50, DSW - 70
    (c) BST - 30, AVL - 70, DSW - 90
    (d) BST - 80, AVL - 80, DSW - 50
    (e) BST - 90, AVL - 90, DSW - 40

30. How many excess rotations are to be performed when balancing the tree using the DSW algorithm?

    (a) 0
    (b) 1
    (c) 2
    (d) 3
    (e) 4

31. What is/are the values of the elements that the excess rotations are performed?

    (a) 05
    (b) 10
    (c) 30
    (d) 50
    (e) 60

32. If node 30 is deleted, what are the possible candidates to take the position of the deleted nodes in the balanced DSW tree?

    (a) 00 and 40
    (b) 05 and 50
    (c) 10 and 40
    (d) 30 and 90
    (e) 80 and 100

PART I

33. What is the total number of rotations performed by the DSW algorithm?

    (a) 06
    (b) 08
    (c) 14
    (d) 16
    (e) 18

34. Which of the following is/are accurate regarding the DSW tree after the deletion of nodes 60, 00, 10 and 40 in that order?

    (a) The resulting tree can be a complete binary tree.
    (b) The resulting tree can be a full binary tree.
    (c) The resulting tree can be a balanced binary tree.
    (d) The resulting tree can be a perfect binary tree.
    (e) The resulting tree can be a degenerate binary tree.

35. If node 50, 40, 80, and 100 are deleted from the above-described AVL tree, which of the following is/are accurate about the resulting AVL tree?

    (a) The resulting tree can be a complete binary tree.
    (b) The resulting tree can be a full binary tree.
    (c) The resulting tree can be a balanced binary tree.
    (d) The resulting tree can be a perfect binary tree.
    (e) The resulting tree can be a degenerate binary tree.

PART II

Answer Question 1 and 2 in the answer book provided.

1. Compute the upperbound complexity of the following code snippet. Your answer should have step by step construction of run-time function (*f(n)*). No credit will be given to answers that only states the complexity. **[20 marks]**

```
i = 1;
for(i = 1; i < n; i = i*3)
{
    //other code
}

for(j = 0; j*j < i; j++)
{
    //other code
}
```

2. Using the Radix Sort algorithm sort the array [160, 46, 74, 92, 807, 241, 12, 66]. Clearly explain and show the intermediate steps. **[10 marks]**

——— *End of Paper* ———